



Atividade 06 — O Esqueleto MVC da Livraria

UC: Programação Back-End | **Bloco 2 — Aula de 19/08/2026**

Formato: EM GRUPO, no repositório do grupo | **Tempo:** 35 minutos

Avaliação: formativa — estrutura de pastas MVC validada por grupo

Objetivo

Montar, em grupo, o esqueleto MVC completo da API da Livraria: servidor rodando, as 4 pastas criadas, um arquivo de referência em cada camada nova, e os models consolidados.

Divisão de trabalho

Quem	Responsável por
Integrante 1	Instalar o Express e deixar o servidor rodando
Integrante 2	Criar as 3 pastas novas e os 3 arquivos de referência
Integrante 3	Consolidar <code>Livro.js</code> em <code>src/models/</code>
Integrante 4	Consolidar <code>Categoria.js</code> em <code>src/models/</code>

Se o grupo tiver menos de 4 integrantes, acumulem tarefas — mas cada um precisa ter feito ao menos um commit próprio hoje.

Parte 1 — Confirmar o servidor rodando (10 min)

```
git pull
npm init -y
npm install express
npm run dev
```

Abram `http://localhost:3000` no navegador. Devem ver a mensagem `API da Livraria no ar!`.

💥 Se aparecer `Cannot find module 'express'` : confira se rodaram `npm install express` dentro da pasta do repositório do grupo.

Parte 2 — Criar o esqueleto de pastas (10 min)

Dentro de `src/` :

```
mkdir routes controllers services
```

Criem os 3 arquivos abaixo, exatamente como estão. Ainda sem lógica de verdade — só a indicação de qual é a função de cada camada.

`src/routes/livroRoutes.js`

```
// ROTA (o "garcom"): recebe a requisicao HTTP.
// Aqui vao ficar os caminhos (endpoints) relacionados a Livro.
// Ex: GET /livros, POST /livros
// Implementacao chega no Bloco 3, quando o banco de dados entrar.

module.exports = {};
```

`src/controllers/livroController.js`

```
// CONTROLLER (o "chef"): decide o que fazer com cada pedido.
// Recebe da rota, chama o service certo, devolve a resposta.
// Implementacao chega no Bloco 3.

module.exports = {};
```

`src/services/livroService.js`

```
// SERVICE (o "cozinheiro"): executa a logica de verdade.  
// Buscar, calcular, validar.  
// Implementacao chega no Bloco 3.
```

```
module.exports = {};
```

Confirmam: `src/models/` já deve existir, com `Pessoa.js`, `Cliente.js` e `Funcionario.js` dentro, desde 07/08.

Parte 3 — Consolidar os models (10 min)

O grupo já tem várias versões de `Livro.js` e `Categoria.js` — uma por integrante, feitas em 05/08 e 12/08 nos repositórios pessoais.

1. Decidam juntos qual versão de cada uma vai para o projeto.
2. Copiem os dois arquivos escolhidos para `src/models/` do repositório do grupo.
3. Confirmam que os arquivos têm `module.exports` no final — sem isso, ninguém mais consegue usá-los.

💡 Não existe versão "errada" — qualquer uma que funcione (roda sem erro, tem `#privado`, `get`, `set`) serve. O critério é qual o grupo inteiro entende melhor.

Parte 4 — Validar a estrutura final

Ao final, a estrutura de `src/` do grupo deve estar assim:

```
src/
├── index.js                ← servidor Express rodando
├── routes/
│   └── livroRoutes.js     ← recebe a requisicao
├── controllers/
│   └── livroController.js ← decide o que fazer
├── services/
│   └── livroService.js    ← executa a logica
└── models/
    ├── Livro.js          ← ja existia, so foi consolidado
    ├── Categoria.js      ← idem
    ├── Pessoa.js         ← ja estava aqui desde 07/08
    ├── Cliente.js        ← idem
    └── Funcionario.js    ← idem
```

Conferir em voz alta, como grupo: para cada arquivo, respondam qual das 4 camadas ele representa e por quê. Se alguém do grupo não souber responder de cabeça, é sinal de rever a seção 3 do guia antes de seguir.

Parte 5 — Enviar

```
git add .
git commit -m "feat: adiciona esqueleto MVC e servidor Express inicial"
git push
```

🔒 Lembrete: cada integrante envia pelo próprio commit. Atualizem também a tabela de responsabilidades do README do grupo.



Se sobrar tempo

A. Replicar o esqueleto para `Categoria`. Criem `categoriaRoutes.js`, `categoriaController.js`, `categoriaService.js`, seguindo exatamente o mesmo padrão dos arquivos de `Livro`.

B. Uma segunda rota no servidor. No `src/index.js`, acrescentem uma segunda rota, só para praticar:

```
app.get("/sobre", (req, res) => {  
  res.send("Livraria SENAI – Trabalho de PBE, turma 1-2026-SESI_DEV_OC_1");  
});
```

Testem em `http://localhost:3000/sobre` .

C. Discussão em grupo. Sem escrever nada: se a API precisasse também gerenciar `Funcionario` (cadastrar, listar, editar), quais 3 arquivos novos vocês criariam, e com que nomes?

✓ Checklist de entrega

- ☐ `express` instalado e listado em `dependencies`
 - ☐ `npm run dev` roda sem erro, mensagem aparece em `http://localhost:3000`
 - ☐ Pastas `routes/` , `controllers/` , `services/` criadas dentro de `src/`
 - ☐ Os 3 arquivos de referência criados, com os comentários corretos
 - ☐ `src/models/` com `Livro.js` , `Categoria.js` , `Pessoa.js` , `Cliente.js` , `Funcionario.js`
 - ☐ O grupo consegue explicar, para cada arquivo, qual das 4 camadas ele representa
 - ☐ Tabela de responsabilidades do README atualizada
 - ☐ `git push` feito, cada integrante com ao menos 1 commit próprio
-

💥 Se der erro

Sintoma	Causa provável	Solução
<code>Cannot find module 'express'</code>	<code>npm install express</code> não rodou na pasta certa	Confira que está dentro de <code>livraria-api-grupoN</code>
<code>EADDRINUSE</code> ao rodar <code>npm run dev</code>	Já tem um servidor rodando em outra janela	Feche o terminal anterior, ou <code>Ctrl + C</code> nele primeiro
<code>Livro is not a constructor</code> , ao testar o model	Esqueceram do <code>module.exports = Livro;</code> no arquivo copiado	Confira a última linha do arquivo

Sintoma	Causa provável	Solução
Pasta <code>models/</code> "sumiu"	Provavelmente não fizeram <code>git pull</code> antes de começar	Rodem <code>git pull</code> para trazer o que já existia desde 07/08